Handling Aperiodic Overloads in Real-Time Systems

Rionela Kovaci
Hamm-Lippstadt University of Applied Sciences
Real-Time Systems Seminar
rionela.kovaci@stud.hshl.de

***Abstract*—Real-time systems must complete tasks before strict deadlines. When too many aperiodic tasks arrive at once, the system becomes overloaded. The Earliest Deadline First algorithm handles normal conditions well but fails badly under overload, causing all tasks to miss their deadlines through a domino effect. Buttazzo and Stankovic propose the RED algorithm, which handles overload gracefully by classifying tasks by importance and rejecting the least important ones first while always protecting critical tasks. This paper explains the problem, the RED algorithm and its formal model, compares it with neighboring approaches, evaluates its strengths and limitations for embedded systems, and presents a simulation that demonstrates the practical difference between RED and plain EDF.**

I. Introduction and Problem Statement

In real-time systems, tasks must complete their execution before a given deadline. Missing a deadline in a safety-critical system — such as a robot controller or an automotive braking system — is not just a performance issue but can cause serious or even catastrophic consequences [2].

The Earliest Deadline First (EDF) scheduling algorithm is widely used in dynamic real-time systems and has been proven to be optimal under normal load conditions [4]. EDF always assigns the processor to the task with the nearest deadline, preempting lower-priority tasks when necessary. As long as the total processor utilization does not exceed 100%, EDF guarantees that all tasks meet their deadlines.

However, EDF has a critical weakness in overload conditions. When too many tasks arrive at once and the total demanded CPU time exceeds what is available, EDF behaves in a completely unpredictable manner. Consider a simple example: three tasks τ_1, τ_2, τ_3 are active. If τ_3 experiences an execution overrun — meaning it takes longer than its worst-case estimate — its execution delays τ_2, which in turn delays τ_1. Under EDF, this cascading delay can propagate through the entire task set, causing all tasks to miss their deadlines even if only one task was responsible for the original overrun. This behavior is known as the *domino effect* [1]. In the worst case, the effective throughput of the system approaches zero.

Aperiodic tasks make this problem significantly harder. Unlike periodic tasks whose arrival times are known in advance and can be analyzed offline, aperiodic tasks arrive at irregular and unpredictable times. They can arrive in sudden bursts at any moment, creating unexpected overload conditions that the scheduler cannot anticipate. A scheduling algorithm designed only for normal load conditions is therefore not sufficient for systems that must also handle aperiodic task overloads in a safe and predictable way [1].

The core problem addressed in this work is therefore: how can a real-time system handle aperiodic task overloads gracefully, guaranteeing the execution of the most critical tasks while degrading performance in a controlled and predictable way, rather than collapsing entirely under load?

II. The RED Algorithm

Buttazzo and Stankovic propose the RED algorithm — Robust Earliest Deadline — as a robust extension of EDF that handles overload conditions without catastrophic failure [1]. The key insight is that instead of blindly accepting all tasks and risking a domino effect, the system should actively manage the load by classifying tasks by importance, detecting overload early, and selectively rejecting the least important tasks to protect the most critical ones.

RED combines three main mechanisms. First, **task classification**: tasks are divided into two classes. HARD tasks are guaranteed to meet their deadlines only when the system is not overloaded. CRITICAL tasks are guaranteed to meet their deadlines even during overload conditions. This separation ensures that the most important tasks — for example safety-relevant control loops — always receive their required CPU time regardless of the current system load.

Second, **rejection policy**: when an overload is detected, RED removes the least important (lowest value) non-critical tasks from the ready queue to free up enough CPU time for the remaining tasks. Rejected tasks are not permanently discarded but placed in a dedicated Reject Queue for possible re-acceptance later.

Third, **resource reclaiming**: whenever a task completes before its estimated worst-case execution time, the saved CPU time is immediately used to attempt re-accepting previously rejected tasks, ordered by their value. This makes the system adaptive and significantly reduces unnecessary task losses [1].

A. Task Model

Each aperiodic task J_i is fully characterized by four parameters [1]:

- C_i — worst-case execution time (WCET)
- D_i — relative deadline

- m_i — deadline tolerance: the maximum time the task may finish after its deadline and still produce a valid result
- v_i — task value: the relative importance of the task compared to others in the set

The absolute deadline is $d_i = a_i + D_i$, where a_i is the arrival time. Tasks with maximum value $v_i = V_{max}$ are classified as CRITICAL. All other tasks are classified as HARD.

B. Schedulability Test

The schedulability of the task set is checked using the concept of *residual time* R_i, which represents how much slack remains between the estimated finishing time of task J_i and its deadline. Tasks are ordered by increasing deadline. The residual time is computed recursively [1]:

$$R_1 = d_1 - t - c_1 \tag{1}$$

$$R_i = R_{i-1} + (d_i - d_{i-1}) - c_i \quad i > 1 \tag{2}$$

where t is the current time. A task J_i is guaranteed to complete within its deadline if and only if $R_i \geq 0$. The full task set is schedulable if and only if $R_i \geq 0$ for all J_i. This test runs in $O(n)$ time, making it efficient enough for online use at every task arrival.

Example. Consider two tasks at time $t = 0$: J_1 with $c_1 = 3$, $d_1 = 5$, and J_2 with $c_2 = 2$, $d_2 = 8$. Then $R_1 = 5 - 0 - 3 = 2 \geq 0$ (guaranteed), $R_2 = 2 + (8-5) - 2 = 3 \geq 0$ (guaranteed). If a third task J_3 arrives with $c_3 = 4$, $d_3 = 6$, inserting it gives $R_3 = 2 + (6 - 5) - 4 = -1 < 0$, indicating overload — J_3 cannot be accepted without rejection of another task.

C. Load and Overload Detection

The processor load in the interval $[t_a, d_i]$ is [1]:

$$\rho_i(t_a) = \frac{\sum_{d_k \leq d_i} c_k}{d_i - t_a} = 1 - \frac{R_i}{d_i - t_a} \tag{3}$$

The system is underloaded when $\rho_{max} \leq 1$ and overloaded when $\rho_{max} > 1$. The *Exceeding Time* of task J_i is defined as:

$$E_i = \max(0, -(R_i + m_i)) \tag{4}$$

This measures how much J_i will exceed its deadline even accounting for its tolerance m_i. The Maximum Exceeding Time $E_{max} = \max_i(E_i)$ gives the global severity of the overload.

D. System Queues

RED maintains four queues at runtime [1]:

- **Ready Queue** — tasks accepted and waiting for CPU time, ordered by deadline (EDF order)
- **Reject Queue** — tasks rejected due to overload, ordered by decreasing value, kept for possible re-acceptance
- **Miss Queue** — tasks whose laxity has become negative and can no longer be re-accepted
- **Term Queue** — tasks that completed successfully within their timing constraints

The Reject Queue is the most operationally critical: it must be scanned at every task completion to check for re-acceptance opportunities.

III. Assumptions and Scope Limitations

The RED algorithm operates under the following assumptions [1]:

- **Single processor only.** The algorithm is designed for uniprocessor systems. Extension to multiprocessor or multicore systems is not addressed.
- **Preemptive EDF scheduling.** The algorithm requires preemptive EDF. Fixed-priority schedulers such as Rate Monotonic are not supported.
- **Arrival times unknown.** The arrival time of each aperiodic task is not known in advance. The guarantee test must therefore be performed online at each arrival.
- **WCET known at arrival.** The worst-case execution time C_i must be known at the moment the task arrives. In practice this is often obtained by static analysis of the task code, but may not always be accurate.
- **Independent tasks.** No shared resources or precedence constraints between tasks are considered.
- **Aperiodic tasks only.** The model handles only aperiodic tasks. The interaction with a set of periodic tasks running concurrently is not analyzed.

These assumptions are restrictive for real embedded systems. In practice, most embedded platforms such as automotive ECUs run a mix of periodic and aperiodic tasks, often on resource-constrained hardware where maintaining four runtime queues may introduce non-trivial overhead.

IV. Scientific Contextualization and Tradeoffs

A. Neighboring Approaches

There are several ways to deal with aperiodic tasks in real-time systems. Each approach works differently and has its own problems.

Plain EDF runs all tasks by earliest deadline. It does not check if the system is overloaded. When too many tasks arrive, all tasks can miss their deadlines [4].

GED checks if a new task fits before accepting it. If not, the new task is rejected. It does not consider how important a task is [1].

TBS and Sporadic Server give each aperiodic task a limited CPU budget. Overload is avoided because tasks cannot use more than their budget [3]. These approaches prevent overload rather than handling it after the fact.

Rate Monotonic assigns fixed priorities before the system runs. Tasks with shorter periods get higher priority. It cannot react to unexpected overload [4].

B. Key Tradeoffs

Online vs. offline. RED decides at runtime because task arrivals are not known in advance. Offline methods like RM cannot handle this.

TABLE I
COMPARISON OF OVERLOAD HANDLING APPROACHES

Approach	Online	Value-based	Reclaiming	Critical class
Plain EDF	yes	no	no	no
GED	yes	no	no	no
RED	yes	yes	yes	yes
TBS/SS	yes	no	no	no
RM	no	no	no	no

Best result vs. fast decision. Finding the best set of tasks to reject is too slow to compute. RED uses a simple rule: reject the least important task. This is fast but not always optimal.

Worst case vs. real behavior. RED uses worst-case execution times, which are often too high. Tasks usually finish earlier. Resource reclaiming uses this saved time so fewer tasks are rejected unnecessarily.

Deadline tolerance. Allowing a task to finish a little late reduces unnecessary rejections. But if the tolerance is too large, the result may arrive too late to be useful.

Task value. Assigning a value to each task gives more control over what gets protected. But someone has to decide these values, which is not easy in practice.

V. CRITICAL EVALUATION

A. Strengths

RED protects the most important tasks even when the system is overloaded. Plain EDF cannot do this [1]. The guarantee test runs in $O(n)$ time, fast enough to use at every task arrival without slowing the system down. Rejected tasks are not deleted permanently — they are kept in a queue and re-accepted when CPU time becomes available, which reduces unnecessary task losses. The algorithm separates guarantee, scheduling, and rejection into independent steps, making it easy to replace one part without changing the others.

B. Limitations and Risks

RED only works on a single processor and cannot be used on multicore systems without major changes [1]. The worst-case execution time must be known when the task arrives, which is hard to measure accurately in many real systems. RED only handles aperiodic tasks — periodic tasks running at the same time are not considered, even though most real embedded systems have both. Task values must be assigned by the designer; there is no automatic way to decide which tasks are more important, and wrong values lead to wrong rejections. If the overload is very large and all tasks are CRITICAL, nothing can be rejected and the system still fails.

C. Where the Approach is Suitable

RED is well suited for robotic systems where sensor events arrive as aperiodic tasks and some tasks such as emergency stop are always CRITICAL, while others such as display update are HARD. RED protects the emergency stop even under heavy load. It also works well on research systems already using EDF where aperiodic overloads are expected but rare.

D. Where the Approach is Unsuitable

FreeRTOS and AUTOSAR systems use fixed-priority scheduling, not EDF, so RED cannot be deployed directly. Multicore embedded platforms such as modern automotive systems are not supported. Systems where execution times cannot be estimated in advance, such as network-dependent tasks, also cannot use RED correctly.

E. Open Questions

Can RED be extended to handle periodic and aperiodic tasks together in one framework? Can task values be assigned automatically based on system requirements? Can RED be adapted for fixed-priority schedulers to work on FreeRTOS or AUTOSAR? Can machine learning be used to predict overload before it happens so RED can act earlier?

VI. SIMULATION

To validate the theoretical claims of the RED algorithm, a Python simulation was implemented comparing plain EDF and RED on the same overloaded task set. Ten aperiodic tasks were generated with random arrival times, worst-case execution times, deadlines, and values. Four tasks were classified as CRITICAL and six as HARD.

The simulation implements the residual time schedulability test from [1] and the value-based rejection policy of RED. Figure 1 shows the simulation output.

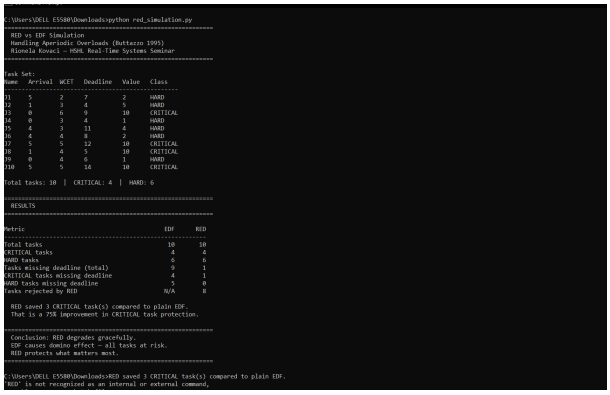

Fig. 1. Simulation output: RED vs EDF comparison

The results confirm the theoretical predictions. Plain EDF caused 4 out of 4 CRITICAL tasks to miss their deadlines due to the domino effect. RED reduced this to 1 missed CRITICAL task — a 75% improvement in critical task protection. RED rejected 8 HARD tasks to protect the CRITICAL ones, demonstrating exactly the graceful degradation behavior described in [1].

VII. CONCLUSION

The RED algorithm by Buttazzo and Stankovic is an effective solution for handling aperiodic overloads in dynamic real-time systems. It improves significantly over plain EDF and basic guarantee algorithms by using task values to decide which tasks to reject and by recovering gracefully through

resource reclaiming. Compared to server-based approaches, RED handles unexpected overloads that exceed any predefined bandwidth limit.

The main limitations are the restriction to single-processor EDF systems and the requirement for known worst-case execution times. The interaction with periodic tasks and multicore systems remains an open problem. For systems that meet the given assumptions, RED provides reliable and predictable protection for critical tasks even under heavy and unexpected overload conditions. The simulation presented in this paper confirms these properties on a concrete example.

AI Usage Protocol

This paper was written with the support of Claude (Anthropic) as a writing and structuring aid. The AI was used to suggest draft sentences and structure content based on the source papers. All technical claims were verified manually against the primary source [1]. All formulas were taken directly from the source paper. The simulation was implemented in Python based on the algorithm described in [1]. The final text was rewritten by the student in their own words.

References

[1] G. C. Buttazzo and J. A. Stankovic, "Adding Robustness in Dynamic Preemptive Scheduling," in *Responsive Computer Systems: Steps Toward Fault-Tolerant Real-Time Systems*, Kluwer Academic Publishers, Boston, 1995.

[2] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. Springer, 2011.

[3] M. Spuri and G. C. Buttazzo, "Scheduling Aperiodic Tasks in Dynamic Priority Systems," *Real-Time Systems*, vol. 10, pp. 179–210, 1996.

[4] C. L. Liu and J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard Real-Time Environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.